

List<String>

<T extends Number & Comparable<T>>

Generic Methods und Types



Object



Generics

Generic Methods

Generische Methoden sind **Methoden**, die **selbst Typparameter definieren** – unabhängig davon, ob die Klasse, in der sie sich befinden, generisch ist oder nicht.

Sie ermöglichen es, Typen flexibel zu verwenden, ohne sich auf einen bestimmten Datentyp festzulegen. Diese Typparameter werden **direkt vor dem Rückgabetyp** der Methode angegeben.

Beispiel: SimpleBag<E>



Mithilfe von Generics ist es möglich, Klassen zu erstellen, die mit einem bestimmten Datentyp arbeiten. Der Bezeichner zwischen den <> kann beliebig gewählt werden.

```
class SimpleBag<E> {  
    private final E element;  
  
    public SimpleBag(E element) {  
        this.element = element;  
    }  
  
    public E getElement() {  
        return this.element;  
    }  
  
    public static void main(String[] args) {  
        SimpleBag<Integer> bagWithNumber = new SimpleBag<>(42);  
        System.out.println(bagWithNumber.getElement());  
  
        SimpleBag<String> bagWithName = new SimpleBag<>("Moin");  
        System.out.println(bagWithName.getElement());  
    }  
}
```

Konvention für Platzhalter:

E - Element (oft in Collections)

K - Key

N - Number

T - Type

V - Value

S, U, etc. für weitere Typen

Ausgabe:

42

Moin

Beispiel: static of



```
class SimpleBag<E> {
```

```
    // Statische Methoden können nicht auf das Generic der Klasse (hier E) zugreifen.
```

```
    // Nicht kompilierbar...
```

```
    static SimpleBag<E> of(E element) {  
        return new SimpleBag<>(element);  
    }
```

```
    // ...  
}
```

Beispiel: static of (Generic Method)



```
class SimpleBag<E> {
```

```
    // Statische Methoden können nicht auf das Generic der Klasse (hier E) zugreifen,  
    // daher wird temporär für die statische Methode of ein Generic T definiert.
```

```
    static <T> SimpleBag<T> of(T element) {  
        return new SimpleBag<>(element);  
    }
```

```
    // ...  
}
```

FYI: Statt T kann auch wieder E für die statische Methode verwendet werden. Der Compiler behandelt dann trotzdem beide E – für die Klasse und Methode – als separate Typen.

Beispiel: map 1/3



```
class SimpleBag<E> {  
  
    private final E element;  
    SimpleBag(E value) { this.element = value; }  
    E getElement() { return element; }  
  
    public static void main(String[] args) {  
        SimpleBag<Integer> bag = new SimpleBag<>(42);  
  
        int elementAsInt = bag.getElement(); // Returns an Integer because <E> of SimpleBag is Integer, which is here autoboxed to int.  
        System.out.println(elementAsInt);  
    }  
}
```

Beispiel: map 2/3



```
class SimpleBag<E> {  
  
    private final E element;  
    SimpleBag(E value) { this.element = value; }  
    E getElement() { return element; }  
  
    public static void main(String[] args) {  
        SimpleBag<Integer> bag = new SimpleBag<>(42);  
  
        int elementAsInt = bag.getElement();  
        System.out.println(elementAsInt);  
  
        double elementAsDouble = bag.map(element -> Double.valueOf(element));  
        System.out.println(elementAsDouble);  
    }  
}
```

Beispiel: map 3/3



```
import java.util.function.Function;
```

```
class SimpleBag<E> {
```

```
    private final E element;
```

```
    SimpleBag(E value) { this.element = value; }
```

```
    E getElement() { return element; }
```

```
    public static void main(String[] args) {
```

```
        SimpleBag<Integer> bag = new SimpleBag<>(42);
```

```
        int elementAsInt = bag.getElement();
```

```
        System.out.println(elementAsInt);
```

```
        double elementAsDouble = bag.map(element -> Double.valueOf(element)); // Double is automatically inferred for R because
        System.out.println(elementAsDouble);                                     // the given lambda returns a Double.
```

```
    }
```

```
    <R> R map(Function<E, R> mapper) { // R depends on the returned type of the given mapper.
        return mapper.apply(element); // E is the same type as the element in the current SimpleBag.
    }                                     // element is an Integer because this SimpleBag<E> stores Integer.
```

```
}
```

```
@FunctionalInterface
interface Function<E, R> {
    R apply(E e);
}
```

Beispiel: map 3/3



```
import java.util.function.Function;
```

```
class SimpleBag<E> {
```

```
    private final E element;
```

```
    SimpleBag(E value) { this.element = value; }
```

```
    E getElement() { return element; }
```

```
    public static void main(String[] args) {
```

```
        SimpleBag<Integer> bag = new SimpleBag<>(42);
```

```
        int elementAsInt = bag.getElement();
```

```
        System.out.println(elementAsInt);
```

```
        double elementAsDouble = bag.map(E: Integerelement -> R: DoubleDouble.valueOf(element)); // Double is automatically inferred for R because
        System.out.println(elementAsDouble); // the given lambda returns a Double.
```

```
    }

    <R> R map(Function<E, R> mapper) { // R depends on the returned type of the given mapper.
        return mapper.apply(element); // E is the same type as the element in the current SimpleBag.
    } // element is an Integer because this SimpleBag<E> stores Integer.
}
```

```
@FunctionalInterface
interface Function<E, R> {
    R apply(E e);
}
```

Erinnerung: Einschränkungen für Generics

Primitive Typen:

Generische Typen können **keine** primitiven Datentypen sein (z. B. <int>, <char>). Stattdessen müssen Wrapper-Klassen (z. B. <Integer>, <Character>) verwendet werden.

Nachteil: **null**-Objekte sind möglich. :-/

Typinformationen zur Laufzeit:

Wegen **Type Erasure** stehen Typparameter zur Laufzeit nicht zur Verfügung. Man kann daher nicht direkt auf den Typparameter zugreifen oder instanziiieren.

Beispiel: **new** T() und obj **instanceof** T funktionieren nicht (es gibt Workarounds).

Weitere Anwendungsfälle mit Generics

Generische Methoden werden typischerweise bei **statischen Methoden** verwendet, da sie nicht auf die generischen Typen der Klasse zugreifen können.

Auch **nicht-statische Methoden** verwenden generischen Typen – häufig im Zusammenspiel mit **Bounds** und/oder für **komplexe Typbeziehungen** (z. B. Invarianz, Kovarianz und Kontravarianz).

Generics: Bounds



Beschränkungen (Bounds) für generische Typen lassen sich mit dem Schlüsselwort **extends** festlegen (gilt auch für generische Klassen).

```
class GenericMethods {  
  
    static <T extends Number> void printAnyNumber(T object) { // object must be of Number or any subtype of Number  
        System.out.println(object);  
    }  
  
    public static void main(String[] args) {  
        printAnyNumber(42); // int  
        printAnyNumber(42.1); // double  
        printAnyNumber("Hallo"); // does not compile, String is not a Number  
    }  
}
```

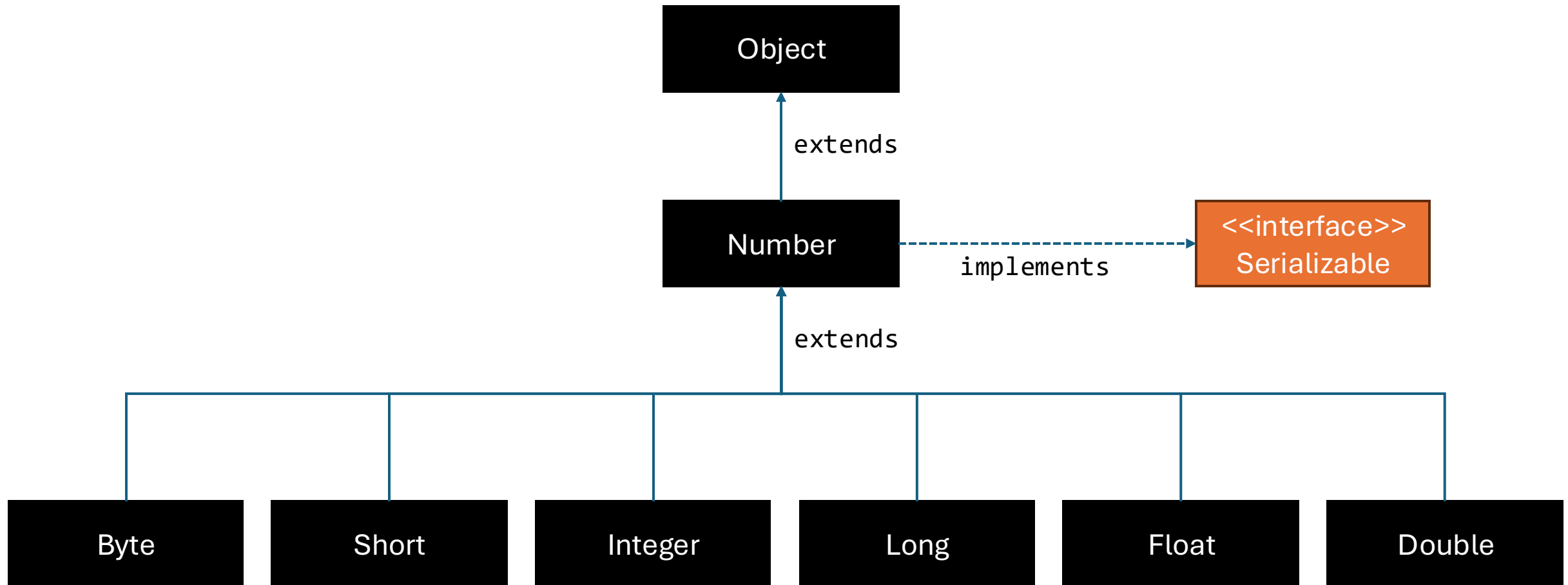
Generics: More Bounds



Weitere **Beschränkungen** für generische Typen lassen sich mit dem **&**-Zeichen ergänzen.

```
class GenericMethods {  
  
    static <T extends Number & Comparable<T>> void printAnyComparableNumber(T object) {  
        System.out.println(object);  
    }  
  
    public static void main(String[] args) {  
        printAnyComparableNumber(42);  
        printAnyComparableNumber(42.1);  
        printAnyComparableNumber("Hallo"); // does not compile, String is not a Number (but it is a Comparable)  
    }  
}
```

Einschub: Klassenhierarchie von Number



Arrays in Java sind *kovariant*.

```
Object[] objectArray = new String[1]; // Das kompiliert!  
objectArray[0] = 42; // ArrayStoreException zur Laufzeit!
```

Da object in Wahrheit auf ein String-Array zeigt, darf ich da eigentlich keine Zahl (42) reinstecken. Der Compiler erlaubt es trotzdem (wegen der Kovarianz). Der Fehler fliegt uns erst zur Laufzeit um die Ohren.

```
List<Object> objectList = new ArrayList<String>(); //  
Kompilierfehler!
```

Compiler: Stopp! Eine Liste von Strings ist keine Liste von Objects, weil du sonst Integers reinstecken könntest!

Generics: Invarianz

Definition: Invarianz bedeutet, dass `GenericType<SubType>` kein Subtyp oder Supertyp von `GenericType<SuperType>` ist – selbst wenn `SubType` ein Untertyp von `SuperType` ist.

→ Ein generischer Typ ist **nur exakt mit dem deklarierten Typ** kompatibel – **nicht mit Unter- oder Obertypen.**

Beispiel:

`ArrayList<Object> numbers = new ArrayList<Integer>();` *// does not compile!*

Generics: Invarianz Beispiel 1

String ist ein **Subtyp** von **Object** (String extends Object).

```
String greetAsString = "Hallo";  
Object greetAsObject = greetAsString;  
System.out.println(greetAsObject);
```

ABER:

List<String> ist **kein Subtyp** von **List<Object>**! *// Invarianz*

Der Grund ist, Typensicherheit zu gewährleisten:

```
List<Integer> listOfIntegers = new ArrayList<>(List.of(1, 2));  
List<Object> listOfObjects = listOfIntegers; // nicht kompilierbar  
listOfObjects.add("Hallo"); // würde einen String in eine List<Integer> hinzufügen.
```

Generics: Invarianz Beispiel 2

Aufgrund von Invarianz wird Vererbung bei Generics standardmäßig nicht berücksichtigt:

```
public static void main(String[] args) {  
    List<Integer> listOfIntegers = new LinkedList<>();  
    List<Long> listOfLongs = listOfIntegers;    // nicht kompilierbar  
    List<Object> listOfObjects = listOfIntegers; // nicht kompilierbar  
  
    // Dieses Problem tritt häufig in Verbindung mit Parametern auf:  
    average(listOfIntegers); // nicht kompilierbar  
}  
  
static double average(List<Number> numbers) { // Integer erbt zwar von Number, aber eine List<Integer>  
    // do calculation...                          wird hier bei der Übergabe nicht akzeptiert.  
                                                    Es werden ausschließlich List<Number> akzeptiert!  
}
```

Generics: Invarianz mit Bounds

Invarianz schränkt die Anwendung von Generics in einer Programmiersprache stark ein. Aber mittels Bounds (Einschränkungen) lässt sich der Anwendungsspielraum erweitern:

```
public static void main(String[] args) {  
    List<Integer> listOfIntegers = new LinkedList<>();  
    List<? extends Long> listOfNumbers = listOfIntegers; // nicht kompilierbar. Integer erbt nicht von Long.  
    List<? extends Object> listOfObjects = listOfIntegers; // kürzer: List<?> listOfObjects  
  
    // Die Methode akzeptiert jetzt listOfIntegers.  
    average(listOfIntegers);  
}  
  
static double average(List<? extends Number> numbers) {  
    // do calculation...  
}
```

Automatische Typinferenz

Invarianz:

→ List<String> ist kein Subtyp von List<Object>.

Der folgende Code ist nicht kompilierbar, da Java automatisch für den Stream den kleinsten gemeinsamen Typ von den Klassen String und Integer herleitet (aka Typinferenz).

In diesem Fall ist das nicht Object, sondern <Serializable & Comparable<? ...>>.

```
List<Object> result = Stream.of("String", 2, "3") // Ursache: Stream mit gemischten Typen.  
    .filter(element -> element.equals("3"))  
    .toList(); // does not compile ❌
```

→ List<Serializable & Comparable<? ...>> ist kein List<Object> (Invarianz).

BTW: Die automatische Typinferenz lässt sich manuell überschreiben:

```
List<Object> result = Stream.<Object>of("String", 2, "3")  
    .filter(element -> element.equals("3"))  
    .toList(); ✅
```

Komplexe Typbeziehungen: Wildcards



Bei Generics können Wildcards **?** verwendet werden, um den generischen Typ offen zu lassen. Es gibt drei Arten von Wildcards:

Unbound

?

```
// Akzeptiert beliebige Listen
void print(List<?> objects) {
    for (Object element : objects) { // Muss Object sein
        System.out.println(element);
    }
}
```

Kovarianz / Upper Bound

? extends Type

```
// Akzeptiert z. B. List<Number>, List<Integer>, List<Double> usw.
double sum(List<? extends Number> numbers) {
    double sum = 0.0;
    for (Number number : numbers) // Der hinzugefügte Wert darf nur
        sum += number.doubleValue(); // Number oder Untertyp sein
    return sum;
}
```

Kontravarianz / Lower Bound

? super Type

```
// Akzeptiert List<Number> oder List<Object>
void addTo(List<? super Number> numbers) {
    numbers.add(3.14); // Kann Number oder Object sein
}
```

Wildcards vs <T>

Ein **Typ-Parameter (T)** definiert eine feste Variable für einen Typen, den du innerhalb der Methode referenzieren musst, um etwa Konsistenz zwischen Parametern und Rückgabewerten zu garantieren.

Eine **Wildcard (?)** steht hingegen für einen unbekannten Typen und wird verwendet, wenn der exakte Inhalt für die Operation irrelevant ist (meistens nur zum Lesen).

Kurz gesagt: Nutze **T**, wenn du den Typen "anfassen" oder zurückgeben musst, und **?**, wenn dir bloße Flexibilität beim Empfangen von Daten reicht.

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) **T mit extends:** wenn der Parameter gelesen und/oder geschrieben wird.

```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
    numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
    for (Object number : numbers) // Jedes Element ist von Typ Object.
        System.out.println(number);
}
```

```
(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```

```
(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```

```
(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) **T mit extends:** wenn der Parameter gelesen und/oder geschrieben wird.

```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
    numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
    for (Object number : numbers) // Jedes Element ist von Typ Object.
        System.out.println(number);
}
```

```
(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```

```
(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```

```
(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) T mit extends: wenn der Parameter gelesen und/oder geschrieben wird.

- ```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
 numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
 for (Object number : numbers) // Jedes Element ist von Typ Object.
 System.out.println(number);
}
```
- ```
(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```
- ```
(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
 numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
 for (Number number : numbers) // Jedes Element ist von Typ Number.
 System.out.println(number);
}
```
- ```
(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) T mit extends: wenn der Parameter gelesen und/oder geschrieben wird.

- ```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
 numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
 for (Object number : numbers) // Jedes Element ist von Typ Object.
 System.out.println(number);
}
```
- ```
(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```
- ```
(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
 numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
 for (Number number : numbers) // Jedes Element ist von Typ Number.
 System.out.println(number);
}
```
- ```
(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) **T mit extends:** wenn der Parameter gelesen und/oder geschrieben wird.

```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
    numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
    for (Object number : numbers) // Jedes Element ist von Typ Object.
        System.out.println(number);
}

(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}

(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}

(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Komplexe Typbeziehungen: Übersicht

Konzept	Beispiel	Erlaubte Zuweisungen
Invarianz	<code>List<Object> list = new ArrayList<Object>();</code>	<code>list</code> akzeptiert ausschließlich <code>List<Object></code>
Kovarianz	<code>List<? extends Number> list = new ArrayList<Integer>();</code> Nur Lesen von <code>list</code> erlaubt. Elemente müssen als <code>Number</code> oder eine Oberklasse von <code>Number</code> gelesen werden.	<code>list</code> akzeptiert <code>List<Number></code> und alle Listen mit Untertypen von <code>Number</code> .
Kontravarianz	<code>List<? super Number> list = new ArrayList<Serializable>();</code> Hinzufügen von <code>Number</code> oder Unterklassen von <code>Number</code> in <code>list</code> erlaubt. Lesen von <code>list</code> erlaubt, aber Elemente müssen als <code>Object</code> gelesen werden.	<code>list</code> akzeptiert <code>List<Number></code> und alle Listen mit Obertypen von <code>Number</code> .
Type-Bounding	<code>static <T extends Number> void print(List<T> list) { ... }</code> <code>static void main(String[] args) {</code> <code>print(new ArrayList<Integer>());</code> <code>}</code> Hinzufügen mit Cast zu <code>(T)(Number)</code> in <code>list</code> erlaubt. Lesen von <code>list</code> erlaubt, aber Elemente müssen als <code>T</code> , <code>Number</code> oder eine Oberklasse von <code>Number</code> gelesen werden.	<code>print</code> akzeptiert <code>List<Number></code> und alle Listen mit Untertypen von <code>Number</code> .



Invarianz / Unbound: Beispiel

```
static void print(List<Object> objects) { // Accepts List<Object> only  
    for (Object element : objects) {  
        System.out.println(element);  
    }  
}
```

```
static void main(String[] args) {  
    List<Integer> numbers = new ArrayList<>();  
    print(numbers); // does not compile, List<Integer> is not a List<Object>  
}
```

Das Wildcard ? wird hier benötigt, um das Programm kompilierbar zu machen:

```
static void print(List<?> objects) // ?-Objekte sind nur lesbar.
```

Alternativ ließe sich dies auch als generische Methode lösen:

```
static <T> void print(List<T> objects)
```



Invarianz / Unbound: Beispiel

```
static void print(List<Object> objects) { // Accepts List<Object> only  
    for (Object element : objects) {  
        System.out.println(element);  
    }  
}
```

```
static void main(String[] args) {  
    List<Integer> numbers = new ArrayList<>();  
    print(numbers); // does not compile, List<Integer> is not a List<Object>  
}
```

Das Wildcard ? wird hier benötigt, um das Programm kompilierbar zu machen:

```
static void print(List<?> objects) // ?-Objekte sind nur lesbar.
```

Alternativ ließe sich dies auch als generische Methode lösen:

```
static <T> void print(List<T> objects)
```



Kovarianz / Upper Bound: Beispiel

Wildcards: Die Methode `printSounds` **akzeptiert** eine Liste von **Pet** oder von einer beliebigen Untertyp von **Pet**.

Kovarianz: Wildcards mit Upper Bounds ermöglichen es, dass die Listen **dogs** und **cats** an `printSounds` übergeben werden können.

Der Compiler garantiert hier, dass `makeSound()` aufgerufen werden kann.

`pets.add(new Dog())` wäre in `printSounds` nicht erlaubt.

```
class PetShop {  
  
    void init() {  
        List<Dog> dogs = List.of(new Dog(), new Dog());  
        List<Cat> cats = List.of(new Cat(), new Cat());  
        List<Pet> pets = List.of(new Dog(), new Cat());  
  
        printSounds(dogs); // subtype list allowed  
        printSounds(cats); // subtype list allowed  
        printSounds(pets); // interface list allowed  
    }  
  
    void printSounds(List<? extends Pet> pets) {  
        for (Pet pet : pets)  
            pet.makeSound();  
    }  
}
```

`printSounds(List<Pet> pets) { ... }`
wäre **nicht** kompilierbar.

```
interface Pet {  
    default void makeSound() {  
        System.out.println("Some generic pet sound");  
    }  
}
```

```
class Cat implements Pet {  
    @Override  
    public void makeSound() {  
        System.out.println("Meow");  
    }  
}
```

```
class Dog implements Pet {  
    @Override  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

Kovarianz / Upper Bound mit final Klassen

Ist eine Klasse final, dann sollte auf das **extends-Bound** verzichtet werden.

Von einer final Klasse kann nicht geerbt werden, d. h. es gibt keine Untertypen.

// Zwei valide Varianten:

```
static void print(List<? extends Integer> integers) { ... }
```

```
static <T extends Integer> void print(List<T> integers) { ... }
```

// Besser:

```
static void print(List<Integer> objects)
```

```
package java.lang;

@jdk.internal.ValueBased
public final class Integer extends Number
    implements Comparable<Integer>, Constable, ConstantDesc {
    // ...
}
```



Kontravarianz / Lower Bound: Beispiel

Wildcards: Die Methode trainDog **akzeptiert** eine Liste von **Dog**- oder von **Pet-Objekten**.

Kontravarianz: Wildcards mit Lower Bounds ermöglichen es, dass die Methode trainDogs eine Liste mit **mindestens den Typ Dog akzeptiert** und es können **nur Dog-Objekte und mögliche Untertyp von Dog** zur Liste hinzugefügt werden.

Die Liste pets kann nur **als Object gelesen** werden.

trainDog(List<Pet> pets) {...}
wäre **nicht** kompilierbar.

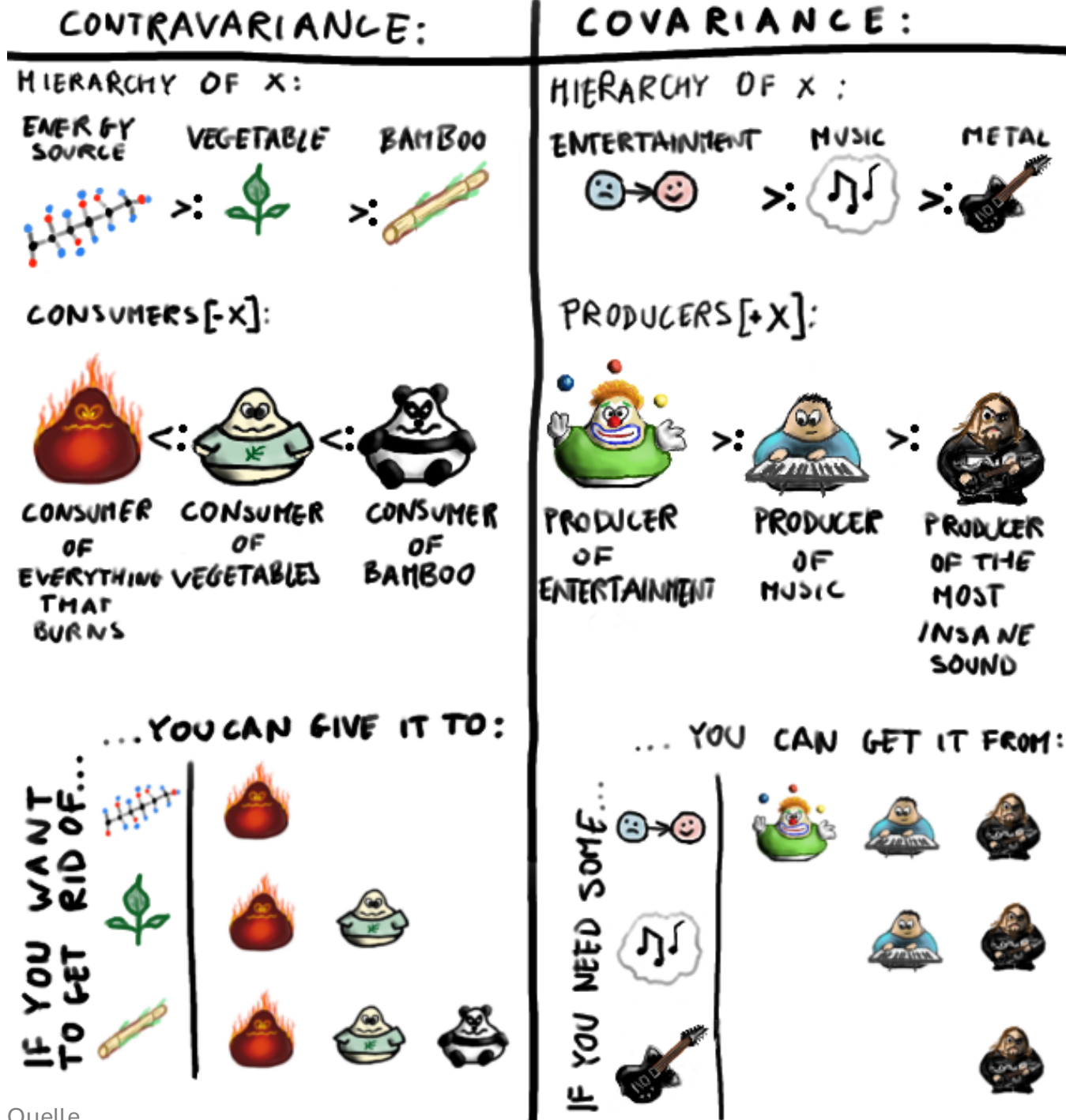
```
class DogTrainer {  
  
    void init() {  
        List<Dog> dogs = new ArrayList<>();  
        dogs.add(new Dog());  
  
        List<Cat> cats = new ArrayList<>();  
        cats.add(new Cat());  
  
        List<Pet> pets = new ArrayList<>();  
        pets.add(new Dog());  
        pets.add(new Cat());  
  
        trainDog(dogs); // subtype list allowed  
        trainDog(cats); // does not compile  
        trainDog(pets); // interface list allowed  
    }  
  
    void trainDog(List<? super Dog> pets) {  
        pets.add(new Dog()); // only Dog is allowed  
        // Read only as Object (and casting) possible  
        for (Object petObj : pets)  
            if (petObj instanceof Pet pet)  
                pet.makeSound();  
    }  
}
```

```
interface Pet {  
    default void makeSound() {  
        System.out.println("Some generic pet sound");  
    }  
}
```

```
class Cat implements Pet {  
    @Override  
    public void makeSound() {  
        System.out.println("Meow");  
    }  
}
```

```
class Dog implements Pet {  
    @Override  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

Kontravarianz vs Kovarianz





PECS (Producer Extends, Consumer Super)

Producer Extends (Upper Bound / Kovarianz)

Verwende ? extends Type, wenn Daten aus einer Struktur **gelesen** werden.
Die Liste numbers "produziert" hier die Elemente.

// Accept List<Number> and all subtypes of Number, e.g., List<Integer>, List<Double> ...

```
void readNumbers(List<? extends Number> numbers) {  
    for (Number number : numbers) {  
        System.out.println(number);  
    }  
}
```

Consumer Super (Lower Bound / Kontravarianz)

Verwende ? super Type, wenn Daten in eine Struktur **geschrieben** werden.
Die Liste numbers "konsumiert" hier die Elemente.

// Accept List<Integer> and any super types of Integer, e.g., List<Number> ...

```
void addIntegers(List<? super Integer> numbers) {  
    numbers.add(1);  
    numbers.add(2);  
    numbers.add(3.3); // does not compile, must be Integer  
}
```

```
public interface Stream<T> {  
    <R> Stream<R> map(Function<? super T, ? extends R> mapper) { ... }  
}
```

B,

```
public static void main(String[] args) {  
    Stream<Integer> intStream = Stream.of(1, 2, 3); // T ist hier Integer  
  
    // Funktion, die Number (Obertyp von Integer) akzeptiert (mehr Flexibilität durch ? super T).  
    Function<Number, String> lambda = number -> String.valueOf(number);  
  
    // lambda wird akzeptiert, weil ? super T in Function es erlaubt.  
    Stream<String> stringStream = intStream.map(lambda);  
  
    // Ohne super T wäre dies nicht erlaubt, also wenn Function<T, ? extends R> wäre.  
}
```

Die Eingabetypen der Funktion (? super T) sind entscheidend, um Obertypen von T zuzulassen. Das erlaubt z. B. diese Nutzung:

```
Function<Object, Dog> objectToDog = obj -> (Dog) obj;  
  
// Funktion akzeptiert Object, obwohl T = Animal
```